

Project Proposal - CI/CD Pipeline Poisoning

Jack Lindberg

June 10, 2026

1 Introduction

Continuous Integration and Continuous Delivery (CI/CD) is the practice of automating the stages that code, and any other kind of software, must pass through from a developer's commit to a running production system. Continuous Integration (CI) means that every change pushed to a shared repository is automatically built and tested, catching integration errors early before they affect end users [1]. Continuous Delivery (CD) is the next step in this process where it automatically prepares validated builds for release. Continuous Deployment takes the final step of pushing every passing change directly to production. Together these stages form a so called pipeline: an automated chain that handles code from an individual developer's commit to the deployment to a running live system (production).

Modern development teams rely heavily on CI/CD to ship software quickly and with confidence. As a consequence, these pipelines have become critical infrastructure. They typically hold production secrets and the authority to modify live systems, often with high-level credentials [2]. This makes them high-value targets and a resource that is important to protect, while still not being able to be airgapped as they need to be able to access the production environment to do their work. An adversary who can influence what the pipeline executes gains the same level of access as the pipeline itself, without ever touching a production system directly [3]. With the correct credentials and a big enough code base these changes can even go undetected for a long time in larger organizations or teams.

Configuration management tools such as Ansible [4] are commonly driven from CI/CD pipelines. An Ansible playbook committed to a repository can be automatically applied to production hosts on every push, using the same SSH credentials the operations team uses. This project targets exactly that and shows the consequences of what access to such a code base can result in.

2 Background

2.1 Supply Chain Attacks and CI/CD

A supply chain attack targets the infrastructure used to build, test, or distribute software rather than the systems directly. By compromising a service hosting the dependencies, for example a build system, a dependency repository, or a deployment pipeline, an attacker's code reaches systems automatically and is automatically trusted, since it arrives through a channel the organization already relies on and trusts [3].

CI/CD pipeline poisoning is a specific technique within this family. An attacker who can introduce malicious code into a pipeline's source repository does not need to exploit any vulnerability in the production system: the pipeline itself carries out the attack on their behalf. The OWASP Top 10 CI/CD Security Risks classifies this as CICD-SEC-4: Poisoned Pipeline Execution [3]. MITRE ATT&CK catalogs it as its own Enterprise Execution technique [5]. This attack is classified as a Direct PPE (d-PPE) when the attacker commits directly to the branch the pipeline tracks, meaning execution is immediate without requiring review [6]. In this CTF that is exactly what we can exploit, since no validation of pushed code takes place.

2.2 Assumptions

Three conditions must hold for a d-PPE attack to succeed [6]:

1. **Automatic execution** - the pipeline executes code from the repository without manual approval for each run.
2. **Attacker write access** - the adversary can modify the code the pipeline will execute.
3. **Privilege gap** - the pipeline agent runs with credentials or network access the attacker does not otherwise possess.

None of these conditions are particularly rare. Pipelines are designed to run automatically [1]; broad write access is common in small teams and pipeline agents typically hold deployment keys that go well beyond what any individual contributor needs [3].

2.3 Why Prevention is Difficult

The core problem with prevention is that write access to source code is a legitimate business need. Restricting it in big organizations slows development without eliminating the risk, since a compromised legitimate account grants the same access anyway [3]. Even with mandatory code review, the organization must trust that reviewers catch malicious changes, which can be as small as a single line in a massive repository [6].

Monitoring pipeline execution for malicious behavior is also difficult. Unlike traditional intrusion detection, there is no clean separation between expected and unexpected activity: the pipeline is supposed to modify systems, spawn processes, and make network connections [3].

2.4 Countermeasures

Several controls reduce the attack surface, though none eliminates the risk on its own. Requiring mandatory code review before changes reach the pipeline-tracked branch makes direct d-PPE from a single compromised account harder. Scoping the credentials available to the pipeline to the minimum needed for its specific task limits the damage if it is exploited [2]. There are many other countermeasures but all of them make the Continuous Integration more and more cumbersome with growing teams and expectations.

3 Exploit

3.1 Attack constraints

The attacker's sole access vector is the Gitea [7] instance exposed on port 3000. They possess a short list of leaked credentials for user accounts in the target organization, one of which grants write access to the Ansible [4] configuration repository. The following are the constraints put in place for the attacker:

- The attacker has no SSH or shell access to the deployment machine.
- The attacker has no direct network access to the flag-holder machine.
- The attacker has no administrative access to the Gitea instance.
- No attempt is made to exploit vulnerabilities in the Gitea software itself.

The goal is to achieve code execution on the flag-holder machine exclusively by injecting a malicious task into the Ansible playbook and waiting for the pipeline to apply it. As the attack would otherwise not really be about CI/CD poisoning but rather something else.

3.2 Attack Stages

3.2.1 Prerequisites

The participant receives a short list of credentials for multiple user accounts in the target organization's Gitea instance. But only one of them will have a guessable password from the leaked list.

3.2.2 Reconnaissance

After authenticating with the correct credentials, the participant browses the Gitea repository to read the Ansible inventory and existing playbook [8]. This reveals the internal hostname and SSH user configured for the flag-holder machine — the information needed to craft a working payload.

3.2.3 Weaponization

With the target environment mapped, the participant adds a malicious Ansible task to the existing playbook. Because this is a d-PPE scenario [6], the commit goes directly to the default branch and is deployed on the flag-holder system on the next pipeline run [3].

3.2.4 Delivery and Exploitation

Delivery is a push of the modified playbook to the Gitea repository. A cron-driven pipeline on the deployment machine polls Gitea every two minutes, pulls any changes and applies the playbook with Ansible. Because the deployment machine holds the SSH key that authorizes connections to the flag-holder, the malicious task executes there with those privileges.

4 Implementation

4.1 Technical Details

The environment is built with Docker Compose and consists of two containers connected by an isolated internal Docker network. Only port 3000 (Gitea) on the deployment container is exposed to the host.

4.1.1 Deployment Container

This container hosts the Gitea instance [7] and runs the pipeline. Its configuration is:

- A Gitea instance on port 3000 with the ansible-config repository and multiple user accounts, one of which has the leaked credentials given to participants.
- A cron job running every two minutes as the unprivileged deploy user that pulls the latest playbook from Gitea and applies it using Ansible [4].
- The deploy user holds an SSH private key authorizing connections to the flag-holder.

4.1.2 Flag-Holder Container

This container runs only an SSH server and holds the flag at `/home/target/flag.txt`. It is reachable exclusively via SSH from the deployment machine using the shared key pair. No port is exposed to the host, making it unreachable by the attacker except through a successfully poisoned pipeline.

4.2 Repository Contents

The project repository will contain:

- This proposal and a final report.
- Instructions for running the environment, including software dependencies.
- All source files for the Docker Compose environment.
- The leaked credential list provided to participants as their starting point.
- A proof-of-concept walkthrough demonstrating the entire chain.

References

- [1] M. Fowler, “Continuous integration,” 2024, accessed: 2026-06-09. [Online]. Available: <https://martinfowler.com/articles/continuousIntegration.html>
- [2] OWASP Foundation, “CICD-SEC-5: Insufficient PBAC (pipeline-based access controls),” 2022, accessed: 2026-06-09. [Online]. Available: <https://owasp.org/www-project-top-10-ci-cd-security-risks/CICD-SEC-05-Insufficient-PBAC>
- [3] —, “CICD-SEC-4: Poisoned pipeline execution,” 2022, accessed: 2026-06-09. [Online]. Available: <https://owasp.org/www-project-top-10-ci-cd-security-risks/CICD-SEC-04-Poisoned-Pipeline-Execution>
- [4] Ansible project contributors, “Introduction to ansible,” 2026, accessed: 2026-06-09. [Online]. Available: https://docs.ansible.com/projects/ansible/latest/getting_started/introduction.html
- [5] Mitre, “Poisoned pipeline execution,” 2026, accessed: 2026-06-09. [Online]. Available: <https://attack.mitre.org/techniques/T1677/>
- [6] Bishop Fox, “Poisoned pipeline execution attacks: A look at CI/CD environments,” 2024, accessed: 2026-06-09. [Online]. Available: <https://bishopfox.com/blog/poisoned-pipeline-attack-execution-a-look-at-ci-cd-environments>
- [7] Team Gitea, “What is gitea?” 2026, accessed: 2026-06-09. [Online]. Available: <https://docs.gitea.com/>
- [8] MITRE, “Reconnaissance, tactic ta0043,” 2026, accessed: 2026-06-09. [Online]. Available: <https://attack.mitre.org/tactics/TA0043/>